

전국 휴게소 맛집 및 여정 정보 큐레이션 시스템

1조 | 박민우, 신민서, 이유진, 류건우, 권민지

Flask 기반 웹 서비스 프로젝트

여행 중 맛있는 휴게소를 쉽고 빠르게 추천하는 플랫폼

Highway Gourmet Road

2025년 12월 22일



목차

1

프로젝트 배경 및 목표

사용자 불편 해소와 경량 웹 서비스 구현 목표

3

주요기능 및 기술 요구사항

기능 및 비기능 요구사항 명시

5

프로그램 시연

배포한 홈페이지 시연

7

DB 구조 및 시스템 아키텍처

주요 개요 및 시스템 흐름

2

팀원 및 역할

팀원별 역할 및 담당 영역

4

화면구성 및 UI/UX 개요

인터페이스 구성 및 각 페이지 안내

6

코드 상세 분석

주요 소스 코드 설명

8

프로젝트 요약 및 향후 계획

프로젝트 소감 및 질문과 답변

2. 팀원 및 역할

프로젝트 관리 및 총괄(PM)

박민우

프로젝트 전반의 일정 관리, 리소스 배분, 팀 커뮤니케이션 총괄 및 최종 발표 담당

기술 아키텍처(PL)

신민서

기술 방향성 설정, 시스템 아키텍처 설계, 핵심 백엔드 로직 개발 리드

AI & API 연동

류건우

Gemini API 연동 및 AI 로직 구성, 외부 API Key 관리 및 최적화 담당

UI/UX 디자인

이유진

UI/UX 디자인 및 프론트엔드 구현, 사용자 경험 개선 및 시각적 요소 담당

데이터베이스 & 배포

권민지

MariaDB 구성 및 클라우드 타입 배포 환경 구축, 데이터베이스 관리 및 안정화 담당

3. 주요 기능 및 기술 요구사항

핵심 기능 (Core Features)

- ▶ 경로 탐색: 주소 입력을 통한 실시간 지도 경로 표시 (Kakao Maps API)
- ▶ 휴게소 큐레이션: 탐색 경로 위에 휴게소 아이콘(식당/편의시설) 배치 및 시각화
- ▶ 상세 정보 제공: 휴게소별 대표 맛집 메뉴, 편의시설(주유소, EV충전소, 약국, 수유실) 안내
- ▶ 카카오 경로 연동: 출발/목적지 입력 시 최적 경로 표시 (kakao.maps.services 활용)

비기능 요구사항 (Non-functional Requirements)

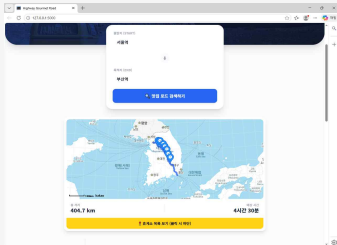
- ▶ 정확도: 카카오 장소 검색 결과와 휴게소 DB 간의 높은 데이터 일치율 유지
- ▶ 성능: 지도 드래그 및 확대/축소 시 끊김 없는 렌더링 성능 확보

구현 체크리스트

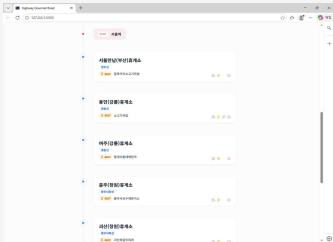
- ✓ Kakao API Key 발급 및 보안 처리
- ✓ 지도 렌더링 최적화 로직 구현
- ✓ 장소 상세 정보 링크 연동 완료

4. 화면 구성 및 UI/UX 개요 - [1]

[1] 출발/도착지 입력

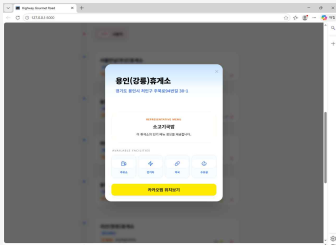


[2] 경로 내 휴게소 목록 출력

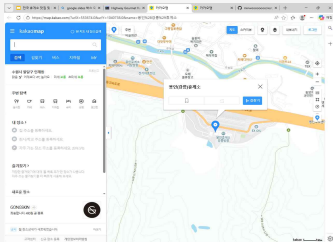


4. 화면 구성 및 UI/UX 개요 - [2]

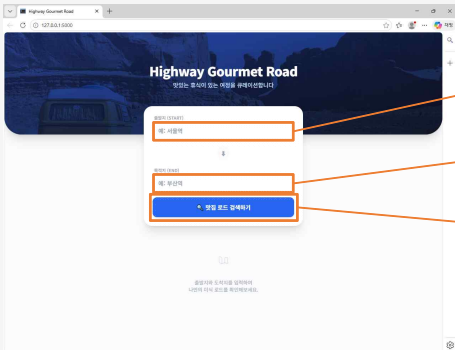
[3] 휴게소 상세 정보



[4] 카카오맵 연동 페이지



5. 프로그램 시연 - [1]

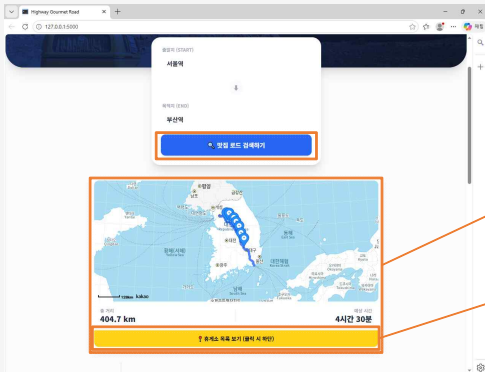


[1] 출발지 입력

[2] 도착지 입력

[3] 경로 내 휴게소 탐색

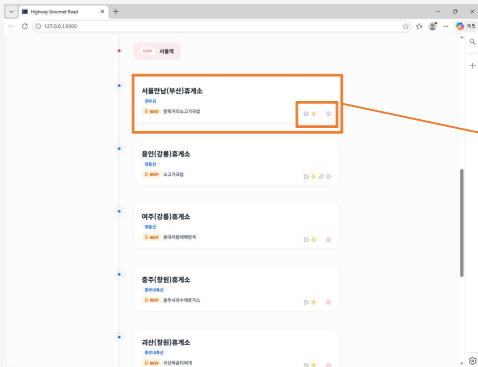
5. 프로그램 시연 - [2]



[4] 지도 내 경로 확인

[5] 휴게소 목록
맨 하단으로 이동

5. 프로그램 시연 - [3]



[6] 휴게소 별 정보 안내

- 도로노선명
- BEST 인기메뉴
- 편의 시설 안내

5. 프로그램 시연 - [4]

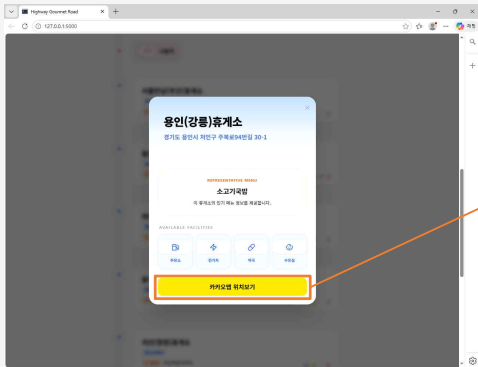
The screenshot shows a web browser window with the address bar displaying 'Highway Gourmet Road' and the URL '127.0.0.1:5900'. A map is visible in the background. A pop-up window titled '용인(강릉)휴게소' (Yongin(Gangneung) Rest Area) is overlaid on the map. The pop-up window contains the following sections:

- 소고기국밥** (Beef Bulgdak): A section with the text '이 휴게소의 인기 메뉴를 소개합니다.' (Introducing the popular menu of this rest area).
- AVAILABLE FACILITIES**: A section with four icons representing different facilities: '주유소' (Gas Station), '편의점' (Convenience Store), '약국' (Pharmacy), and '주유장' (Fuel Station).
- 카카오맵 위치보기** (View Location on KakaoMap): A yellow button at the bottom of the pop-up window.

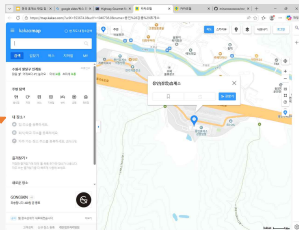
Arrows point from the following elements in the pop-up window to callout boxes on the right:

- From the '소고기국밥' section to the box labeled '[7] BEST 메뉴 안내'.
- From the 'AVAILABLE FACILITIES' section to the box labeled '[8] 편의 시설'.
- From the '카카오맵 위치보기' button to the box labeled '[10] 카카오맵 이동'.

5. 프로그램 시연 - [5]



[11] 카카오맵 이동



6. 코드 상세 분석

Flask 애플리케이션의 핵심 5개 파일

파일 트리

1218_REST_AREA-MAIN

static/

main.js

style.css

templates/

index.html

app.py

db.py

파일 역할

main.js

카카오 지도 렌더링, 출발지/도착지 자동완성, 경로 요청, 휴게소 목록 렌더링

프론트엔드 로직

style.css

전체 UI 스타일, 카드/모달/타임라인 레이아웃 정의

UI 스타일

index.html

메인 화면 구조, 입력 UI, 지도 영역, 휴게소 리스트 모달

HTML 템플릿

app.py

Flask 메인 서버, URL 라우팅, 외부 API 연동 (Kakao, Gemini)

백엔드 컨트롤러

db.py

MariaDB 연결 관리, 휴게소 정보 조회 및 데이터 처리

데이터 계층

6. 코드 상세 분석

App.py

▶ **geocode(query)** 주소/장소를 좌표로 변환

역할 텍스트 주소를 위도 / 경도 좌표로 변환

동작 로직

- [1] Kakao Local API(search/address.json) 을 호출하여 검색
- [2] 결과가 없을 경우 search/keyword.json으로 키워드 재검색
- [3] 성공 시(X, Y) 좌표 반환, 실패 시 ValueError 발생

▶ **get_route(origin, dest)** 경로 데이터 생성

역할 출발지 / 목적지 사이의 추천 경로값 리스트 생성

동작 로직

- [[1] Kakao Mobility API(directions)에 origin, dest, priority = "RECOMMEND" 전송
- [2] 응답 JSON에서 sections -> roads -> vertexes 파싱
- [3] 지도에 그릴 수 있는 (latitude, longitude) 튜플 리스트로 변환

```
# =====  
# 주소 / 장소 -> 좌표 변환  
# =====  
def geocode(query: str):  
    headers = {"Authorization": f"KakaoAK {REST_API_KEY}"}  
    # 주소 검색  
    r = requests.get(  
        "https://dapi.kakao.com/v2/local/search/address.json",  
        headers=headers,  
        params={"query": query},  
        timeout=10  
    ).json()  
  
    if r.get("documents"):  
        d = r["documents"][0]  
        return float(d["x"]), float(d["y"])  
  
    # 키워드 검색  
    r = requests.get(  
        "https://dapi.kakao.com/v2/local/search/keyword.json",  
        headers=headers,  
        params={"query": query},  
        timeout=10  
    ).json()  
  
    if r.get("documents"):  
        d = r["documents"][0]  
        return float(d["x"]), float(d["y"])  
  
    raise ValueError("주소를 좌표로 변환할 수 없습니다.")
```

```
# =====  
# 카카오 모빌리티 길찾기  
# =====  
def get_route(origin, dest):  
    url = "https://apis-navi.kakao mobility.com/v1/directions"  
    headers = {"Authorization": f"KakaoAK {REST_API_KEY}"}  
    params = {  
        "origin": f"{origin[0]},{origin[1]}",  
        "destination": f"{dest[0]},{dest[1]}",  
        "priority": "RECOMMEND"  
    }  
  
    res = requests.get(url,  
        headers=headers,  
        params=params,  
        timeout=15).json()  
  
    routes = res.get("routes", [])  
    if not routes:  
        raise ValueError("길찾기 결과가 없습니다.")  
  
    sections = routes[0].get("sections", [])  
    if not sections:  
        raise ValueError("길찾기 sections가 없습니다.")  
  
    roads = sections[0].get("roads", [])  
    coords = []  
  
    for road in roads:  
        v = road.get("vertexes", [])  
        for i in range(0, len(v), 2):  
            coords.append((v[i], v[i + 1]))  
  
    return coords
```

6. 코드 상세 분석

App.py

- ▶ `@app.route("/route", methods=["POST"])` : 메인 로직

Trigger 사용자가 '검색하기' 버튼 클릭 시

처리 순서

- [1] geocode() : 출발지(start), 목적지(end)를 좌표로 변환
- [2] get.route() : 경로 좌표 데이터(route_points) 생성
- [3] db.get_rest_areas() : DB에서 휴게소 목록 조회
- [4] Response: 경로(route)와 휴게소 데이터(rests)를 JSON으로 변환

- ▶ `@app.route("/get_info", methods=["POST"])` : AI 연동

Trigger 사용자가 지도 상의 휴게소 상세 정보 요청 시

처리 순서

- [1] Google Gemini API 호출(gemini-2.5-flash-lite)
- [2] Prompt: "해당 휴게소를 찾아서 실제로 판매중인 대표메뉴 추천 받기"
- [3] Response: AI가 생성한 텍스트 설명을 JSON으로 변환

```
# =====  
# 페이지  
# =====  
  
@app.route("/")  
def index():  
    return render_template("index.html")  
  
# =====  
# 경로 + 휴게소 조회  
# =====  
  
@app.route("/route", methods=["POST"])  
def route():  
    data = request.get_json(force=True)  
    start = data.get("start")  
    end = data.get("end")  
  
    try:  
        start_xy = geocode(start)  
        end_xy = geocode(end)  
        route_points = get_route(start_xy, end_xy)  
  
        # 🔥 MariaDB에서 휴게소 조회  
        rests = db.get_rest_areas()  
  
        return jsonify({  
            "route": route_points,  
            "rests": rests  
        })  
  
    except Exception as e:  
        return jsonify({"error": str(e)}), 500
```

```
# =====  
# Gemini 휴게소 메뉴 설명  
# =====  
  
@app.route("/get_info", methods=["POST"])  
def get_rest_area_info():  
    try:  
        data = request.get_json()  
        rest_name = data.get("name")  
  
        api_key = "GEMINI_API"  
        url = f"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash-lite:generateText:  
        payload = {  
            "contents": [{  
                "parts": [{  
                    "text": f"{rest_name} 해당 휴게소를 찾아서 실제로 판매중인 대표 메뉴 2개만 알려줘. 중  
                }]  
            }]  
        }  
  
        res = requests.post(url, json=payload, timeout=60).json()  
  
        if "candidates" in res:  
            text = res["candidates"][0][{"content": [{"parts": [{"text":  
            return jsonify({"info": text})  
  
        return jsonify({"error": "AI 응답 오류"}), 500  
  
    except Exception as e:  
        return jsonify({"error": str(e)}), 500  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", debug=True)
```

6. 코드 상세 분석

Main.js

초기화 (window.onload)

설명

- [1] Kakao.maps.Map : 지도 객체 생성 및 초기 중심 좌표 설정
- [2] 이벤트 리스너 등록 : addInputListeners(), setup OutsideClick() 호출

자동완성(autoComplete)

Trigger 사용자가 입력창에 글자를 입력 시

처리 순서

- [1] API : kakao.maps.services.Places().keyword Search() 사용
- [2] UI : 검색 결과를 리스트(DIV)로 생성하여 입력창 아래에 표시 및 클릭 시 해당 장소명 입력

```
// =====  
// 초기화 및 이벤트 바인딩  
// =====  
window.onload = function () {  
  const container = document.getElementById("map");  
  map = new kakao.maps.Map(container, {  
    center: new kakao.maps.LatLng(36.5, 127.6),  
    level: 13,  
  });  
  
  addInputListeners();  
  setupOutsideClick();  
}  
  
function addInputListeners() {  
  const startInput = document.getElementById("start");  
  const endInput = document.getElementById("end");  
  startInput.addEventListener("input", () => autoComplete("start"));  
  endInput.addEventListener("input", () => autoComplete("end"));  
}  
  
function setupOutsideClick() {  
  document.addEventListener("click", (e) => {  
    ["start", "end"].forEach((type) => {  
      const input = document.getElementById(type);  
      const box = document.getElementById(`autocomplete-${type}`);  
      if (input && box && input.contains(e.target) && box.contains(e.target)) {  
        box.classList.add("hidden");  
      }  
    });  
  });  
}
```

```
function autoComplete(type) {  
  if (!isSelectingAutocomplete) return;  
  const input = document.getElementById(type);  
  const keyword = input.value.trim();  
  const box = document.getElementById(`autocomplete-${type}`);  
  
  if (keyword) { box.classList.add("hidden"); return; }  
  
  const ps = new kakao.maps.services.Places();  
  ps.keywordSearch(keyword, (data, status) => {  
    if (status !== kakao.maps.services.Status.OK) { box.classList.add("hidden"); return; }  
    box.innerHTML = "";  
    box.classList.remove("hidden");  
    data.forEach((place) => {  
      const div = document.createElement("div");  
      div.className = "p-3 hover:bg-blue-50 cursor:pointer border-1 border-gray-50 text-border-0";  
      div.innerHTML = `<div class="font-bold text-sm text-gray-600">${place.place_name}</div>`;  
      div.onmousedown = (e) => {  
        e.preventDefault();  
        isSelectingAutocomplete = true;  
        input.value = place.place_name;  
        box.classList.add("hidden");  
        setTimeout(() => { isSelectingAutocomplete = false; input.blur(); }, 50);  
      };  
      box.appendChild(div);  
    });  
  });  
}
```


6. 코드 상세 분석

Main.js

▶ 경로 데이터 요청(requestRoute)

설명

- [1] fetch("/route", POST) : 서버에 출발 / 도착지 전송
- [2] 성공 시 drawRoute(data) 호출, 실패 시 에러 메시지 출력

▶ 지도 그리기(drawRoute)

설명

- [1] Polyline 생성 : 서버에서 입력 받은 좌표 배열을 kakao.maps.LatLng 객체로 변환 후 파란색 선으로 표시
- [2] 화면 조정 : map.setBounds(bounds)를 통해 경로 전체가 보이도록 줌 레벨 고정
- [3] 정보 계산 : calculateTotalDistance (총 거리) 및 estimateTime (예상 소요 시간) 계산 후 표시
- [4] 휴게소 처리 : drawRestAreas(data.rests) 호출하여 휴게소 로직 시작

```
// =====  
// 경로 입력 및 그리기  
// =====  
function requestRoute() {  
  const start = document.getElementById("start").value.trim();  
  const end = document.getElementById("end").value.trim();  
  if (!start || !end) { alert("출발지와 목적지를 모두 입력해주세요."); return; }  
  
  const btn = document.querySelector("button[onclick='requestRoute()']");  
  btn.innerText = "▶ 경로 입력 ...";  
  btn.disabled = true;  
  
  fetch("/route", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ start, end }),  
  })  
    .then((res) => res.json())  
    .then((data) => {  
      btn.innerText = "❌, 맵핑 로드 중입니다";  
      btn.disabled = false;  
      document.getElementById("empty-state").classList.add("hidden");  
      document.getElementById("result-area").classList.remove("hidden");  
      map.relayout();  
      drawRoute(data);  
    })  
    .catch((err) => {  
      btn.innerText = "❌, 맵핑 로드 중입니다";  
      btn.disabled = false;  
      alert("오류: " + err.message);  
    });  
}
```

```
function drawRoute(data) {  
  const path = data.route.map((p) => new kakao.maps.LatLng(p[1], p[0]));  
  if (polyline) polyline.setMap(null);  
  polyline = new kakao.maps.Polyline([ path, strokeWeight: 6, strokeColor: "blue" ],  
    polyline.setMap(map);  
  
  const bounds = new kakao.maps.LatLngBounds();  
  path.forEach((p) => bounds.extend(p));  
  map.setBounds(bounds);  
  
  const totalMeters = calculateTotalDistance(path);  
  document.getElementById("route-meta").classList.remove("hidden");  
  document.getElementById("meta-distance").textContent = `${(totalMeters / 1000).toFixed(1)} km`;  
  document.getElementById("meta-time").textContent = estimateTime(totalMeters);  
  
  lastRests = data.rests || [];  
  drawRestAreas(lastRests);  
}
```

6. 코드 상세 분석

Main.js

휴게소 필터링(drawRestAreas)

설명

- [1] 근접성 체크: isRestAreaNearRoute 함수로 경로 선상 반경 1.5km 이내의 휴게소만 필터링
- [2] 방향 체크: getTravelDirection 함수로 상행 / 하행 여부 판별하여 진행 방향에 맞는 휴게소만 표시
- [3] 정렬: 출발지 기준 거리순으로 오름차순 정렬(filtered.sort)
- [4] 마커 표시: 필터링된 휴게소 위치에 지도 마커 생성
- [5] 정보 카드: 하단 패널에 휴게소 이름, 편의시설 아이콘이 포함된 카드 동적 생성

```
// ----- [1] 휴게소 위치 판별] -----
// 휴게소 코드 및 카드 생성 (행정 지역 / highway_name 반영)
// -----
function drawRestAreas(rests) {
  const list = document.getElementById("rest-list");
  list.innerHTML = "";
  markers.forEach((m) => m.setMap(null));
  markers = [];

  if (!polyline) return;
  const path = polyline.getPath();
  const travelDirection = getTravelDirection(path);
  const startPoint = path[0];

  // 0. 출발지 및 도착지 여백 가지우기
  const startName = document.getElementById("start").value.trim();
  const endName = document.getElementById("end").value.trim();

  let filtered = rests.filter((r) => isRestAreaNearRoute(r.lat, r.lng, path));

  filtered.sort((a, b) => getDistance(startPoint, a), startPoint, b);

  // --- [1] 필터링된 휴게소 카드 추가 ---
  const startItem = document.createElement("div");
  startItem.className = "timeline-item animate-fade-in-up";
  startItem.innerHTML = `
    <div class="timeline-dot bg-blue-600 border-red shadow-sm"></div>
    <div class="timeline-card-wrapper flex">
      <div class="bg-red-300/20 p-3 px-4 rounded-xl border border-red-200">
        <span class="text-[9px] font-bold text-red-600 uppercase tracking-wider">
          <div>
            </div>
          </div>
        <div class="text-sm font-bold text-gray-900"><div>
            </div>
          </div>
        </div>
      </div>
    `;
  list.appendChild(startItem);
}
```

```
// ----- [2] 휴게소 카드 [정보] -----
filtered.forEach((r, idx) => {
  const loc = new kakao.maps.LatLng(r.lat, r.lng);
  const marker = new kakao.maps.Marker({ position: loc, map: map });
  markers.push(marker);
  kakao.maps.event.addListener(marker, "click", () => openSimpleInfo(marker));

  const factNames = `
    <div class="absolute bottom-4 right-0 flex gap-1.5">
      <div data-locid="fuel" class="w-3.5 h-3.5 ${r.gas === 'Y' ? 'text-green-500' : 'text-gray-500'}"></div>
      <div data-locid="map" class="w-3.5 h-3.5 ${r.alms === 'Y' ? 'text-blue-500' : 'text-gray-500'}"></div>
      <div data-locid="pill" class="w-3.5 h-3.5 ${r.pharmacy === 'Y' ? 'text-red-500' : 'text-gray-500'}"></div>
    </div>
  `;

  const item = document.createElement("div");
  item.className = "timeline-item animate-fade-in-up";
  item.style.animationDelay = ` ${idx + 1} * 0.1s`; // 출발지 이후부터 순차적

  item.innerHTML = `
    <div class="timeline-dot"></div>
    <div class="timeline-card-wrapper">
      <div class="relative bg-white p-5 pr-10 rounded-xl border border-gray-100">
        <div class="font-black text-[9px] text-gray-800 mb-1">[formatRestName]
        <div class="flex items-center mb-2">
          <div class="text-[10px] font-bold text-blue-600 bg-blue-50 p-2">
            [r.highway_name] [고속도로]
          </div>
        </div>
        <div class="flex items-center gap-1.5">
          <div class="flex items-center gap-1.5">
            <div data-locid="fuel" class="w-3 h-3"></div>
            <div data-locid="map" class="w-3 h-3"></div>
            <div data-locid="pill" class="w-3 h-3"></div>
          </div>
          <div class="text-sm font-bold text-gray-600 truncate max-w-[100px]">
            [r.food] [대표 메뉴 정보]
          </div>
        </div>
      </div>
    </div>
  `;
  list.appendChild(item);
}
```

6. 코드 상세 분석

Main.js

▶ 상세 정보 모달(openRestModal)

설명

- [1] Trigger : 휴게소 카드 또는 마커 클릭 시 실행
- [2] 역지오코딩(Client-side) : getAddressFromCoords 함수로 좌표를 주소(텍스트)로 변환하여 표출(서버 부하 감소 목적)
- [3] Kakao Map 연동 : '위치 보기' 버튼 클릭 시 map.kakao.com 새창 열기
- [4] AI 메뉴 추천 : fetch("/get_info") 호출하여 휴게소 이름 전송
- [5] Gemini 응답을 받아 "정보 불러오는 중.." 텍스트를 AI가 작성한 메뉴 설명으로 교체

```
window.handleClick = function (restId) {
  const r = window.restData?.[restId];
  if (r) openRestModal(r);
};

function openRestModal(rest) {
  const restName = formatRestName(rest.name);
  document.getElementById("modal-name").textContent = restName;
  const addrEl = document.getElementById("modal-address");

  if (addressCache[rest.id]) addrEl.textContent = addressCache[rest.id];
  else getAddressFromCoords(rest.lat, rest.lng, (addr) => {
    addressCache[rest.id] = addr;
    addrEl.textContent = addr;
  });

  document.getElementById("modal-menu-name").textContent = rest.food || "정보";

  setFac("fac-gas", rest.gas);
  setFac("fac-ev", rest.elec);
  setFac("fac-pharmacy", rest.pharmacy);
  setFac("fac-baby", rest.nurse);

  // --- [추가된 부분: 카카오맵 위치보기 버튼 로직] ---
  const kakaoBtn = document.getElementById("modal-kakao");
  kakaoBtn.onclick = () => {
    // 카카오맵 위치 찾기 공유/일정기 URL (좌표와 장소명 전달)
    const url = "https://map.kakao.com/link/map/${restName},${rest.lat},${rest.lng}";
    window.open(url, "_blank"); // 새 창에서 열기
  };

  const descEl = document.getElementById("modal-menu-desc");
  descEl.textContent = "정보를 불러오는 중...";
  fetch("/get_info", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: rest.name })
  })
  .then(res => res.json())
  .then(data => {
    descEl.innerHTML = data.info ? data.info.replace(/\n/g, "<br>") : "이";
  })
  .catch(() => descEl.textContent = "정보를 불러오지 못했습니다.");

  document.getElementById("rest-modal").classList.remove("hidden");
}
```

6. 코드 상세 분석

db.py

클래스 기반 DB 연결 (class Database)

설명

- [1] 초기화(__init__): pymysql을 사용해 MariaDb에 연결 후 os.getenv 파일을 사용해 DB 접속 정보를 유연하게 관리 및 에러 메시지 출력
- [2] 커서 설정: DictCursor를 사용해 쿼리 결과 데이터를 딕셔너리 형태로 반환하여 프론트엔드에 데이터를 보내는데 효율적으로 사용

```
class Database:
    def __init__(self):
        self.connection = None
        try:
            self.connection = pymysql.connect(
                host=os.getenv("DB_HOST", "localhost"),
                port=int(os.getenv("DB_PORT", 3306)),
                database=os.getenv("DB_NAME", "highway"),
                user=os.getenv("DB_USER", "root"),
                password=os.getenv("DB_PASSWORD", "1234"),
                charset="utf8mb4",
                cursorclass=pymysql.cursors.DictCursor
            )
            print("MariaDB에 성공적으로 연결되었습니다.")
        except Error as e:
            print(f"MariaDB 연결 중 오류 발생: {e}")
```

휴게소 전체 조회 (get_rest_areas)

설명

- [1] 데이터 정제: SQL 쿼리 내 IFNULL, AS(Alias)를 사용해 DB의 컬럼명을 앱의 인터페이스(lat, lng, food 등)에 맞게 변환
- [2] 정보 통합: 휴게소 이름 뿐만이 아닌 위도/경도, 노선 번호, 대표 음식, 편의시설 유무(주유소, 전기차충전소, 약국, 수영장)를 한번에 조회

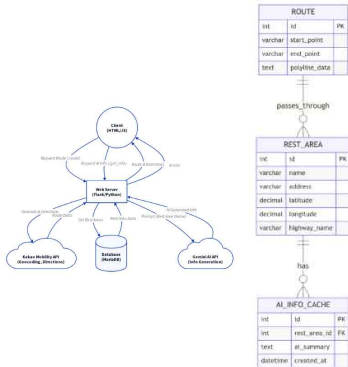
```
def get_rest_areas(self, limit=None):
    """
    rest_areas 테이블에서 휴게소 목록 조회
    """
    if self.connection is None:
        print("데이터베이스 연결이 없습니다.")
        return []

    try:
        with self.connection.cursor() as cursor:
            query = """
            SELECT
                id,
                name,
                route_no,
                direction,
                latitude AS lat,
                longitude AS lng,
                IFNULL(signature_food, '') AS food,
                gas_ys AS gas,
                elec_ys AS elec,
                phar_ys AS pharmacy,
                nursing_ys AS nurse,
                signature_food,
                tel,
                highway_name
            FROM highway
            """
            if limit:
                query += " LIMIT %s"
                cursor.execute(query, (limit,))
            else:
                cursor.execute(query)

            return cursor.fetchall()

    except Error as e:
        print(f"휴게소 조회 중 오류 발생: {e}")
        return []
```

7. DB구조 및 시스템 아키텍처



구조 개요

- ▶ **Framework** Python Flask (백엔드 로직 및 라우팅 처리)
- ▶ **Front-end** HTML5, Tailwind Css, JavaScript (Jinja2 템플릿 엔진)
- ▶ **API** Kakao Maps API, 공공데이터 API
- ▶ **Database** MariaDB

시스템 흐름

- [1] 사용자가 브라우저에 접속
- [2] Flask 서버가 index.html을 Jinja2로 렌더링
- [3] JavaScript가 Kakao Maps API 호출 및 경로 표시
- [4] DB의 휴게소 데이터를 마커로 시각화

8. 프로젝트 요약 및 향후 계획

프로젝트 요약

- Kakao Maps API를 활용한 국내 고속도로 최적화 여정 정보 시스템 구현 완료
- 휴게소 맛집과 상세 정보를 경로상에서 즉시 확인하는 차별화된 경험 제공

향후 계획 (Future Plans)

- 실시간 정보 강화: 카카오맵 내 실시간 혼잡도나 주차 정보 추가 연동
- 개인화 추천: 사용자의 선호 음식(한식/일식 등)에 따른 맞춤형 휴게소 추천 기능 추가
- 내비게이션 연동: 실제 카카오내비 앱으로 경로를 전송하는 기능 추가

8. 프로젝트 요약 및 향후 계획

프로젝트 관리 및 총괄(PM)

박민우

"이번 프로젝트에서 팀원 간의 다양한 의견을 조율해 개발 방향을 제시하고 의사결정을 지원하며 협업의 기술을 배웠습니다. 특히 직접 UI/UX 수정 작업을 수행하며 데이터가 사용자에게 전달되는 최종 접점을 최적화하는 과정을 통해, 서로 다른 기술 스택이 유기적으로 맞물린 때 비로소 완성도 높은 서비스가 탄생한다는 점을 깊이 느꼈습니다. 이를 통해 기술적 조율과 UX 최적화를 아우르는 통합 관리 역량의 중요성을 깨달았습니다."

기술 아키텍처(PL)

신민서

"사용자가 정보를 찾는 수고를 최소화할 수 있도록 Flask 기반의 데이터 처리와 카카오톡 API를 효율적으로 연동했습니다. 프로젝트의 전 과정을 리딩하며 팀의 비전을 실제 서비스로 안착시킨 경험은 PL로서 한 단계 성장하는 소중한 계기가 되었습니다."

UI/UX 디자인

이유진

"Flask가 생소해 어려움도 있었지만, 코드가 실제 화면으로 실현되는 과정이 흥미로웠습니다. 사용자가 더 편리하게 사용할 수 있는 화면을 만드는 데 집중했고, 그 과정에서 전체적인 코드 흐름까지 익힐 수 있었습니다."

AI & API 연동

류건우

"Gemini, 카카오 댄, 카카오 모빌리티 api를 실제 프로젝트에 적용하면서, api key 관리와 예외 처리 등 안정적인 기능 구성의 중요성을 경험했습니다. 단순히 기능 구현을 넘어, 데이터 처리와 프로젝트 설계의 중요성을 배웠습니다"

데이터베이스 & 배포

권민지

"Flask 구조를 직접 설계하고 카카오 api를 실제 적용하면서 파이썬 기반 기술에 대해 명확히 이해할 수 있어 의미 있고 새로운 경험이었습니다. 협업 과정에서는 Git을 활용해 코드 변경 사항을 관리하는 방법을 이해하였고, Git 사용 능력 또한 향상되었습니다."

Q&A

질문과 답변

<http://192.168.0.40:5000/>

감사합니다.